

```

SecurityContext context = new SecurityContextImpl();
//拿到原始请求对象
ServerHttpRequest request = exchange.getRequest();
//如果符合路径
if(request.getURI().getPath().equals("user-center/noMerge")){
    //查询该请求1秒内的qps
    Integer count = (Integer) redisTemplate.opsForValue().get("getUser");
    //如果流量过高超过特定值就要转向合并请求
    //流量过高需要转向合并请求
    if(count >= 100){
        //修改原先请求路径
        String newPath = "user-center/getMergeUser";
        ServerHttpRequest newRequest = request.mutate()
            .path(newPath)
            .build();
        return chain.filter(exchange.mutate().request(newRequest).build()).contextWrite
            (ReactiveSecurityContextHolder.withSecurityContext(Mono.just(context)));
    }
    //流量计数加1
    count++;
    redisTemplate.opsForValue().set("getUser", count);
}
}

```

从监控特定请求的 qps 开始

如根据用户的 userid 获取用户信息请求被指定为特定请求

则在权限过滤器中判断请求路径是否是该请求路径

如果是则查询该请求的这一秒内的 qps 值

若大于特定值如代码里写死的 100

则说明 qps 过高需要转换请求路径到合并请求的路径。

```

@Component 新 *
public class QPSScheduled {
    public RedisTemplate redisTemplate; 1个用法
    //每隔1s执行一次的定时任务
    @Scheduled(fixedRate = 1000) 新 *
    public void initQPS(){
        //将获取用户信息请求接口的qps值重置为0
        redisTemplate.opsForValue().set("getUser", 0);
    }
}

```

同时有定时任务每隔 1s 清零请求的 qps 值

```

package ljl.bilibili.user_center.service;

import io.swagger.annotations.ApiOperation;
import ljl.bilibili.entity.user_center.user_info.User;
import ljl.bilibili.util.Result;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import javax.annotation.Resource;

@RestController
@RequestMapping("⊕"/"user-center")
public class MergeRequestController {
    @Resource 2个用法
    UserWrapBatchService userWrapBatchService;
    @ApiOperation("会合并请求的用户信息查询接口")
    @GetMapping("⊕"/"getMergeUser")
    public Result<User> merge(Integer userId) {
        return Result.data(userWrapBatchService.queryUser(userId));
    }
    @ApiOperation("不会合并请求的用户信息查询接口")
    @RequestMapping("⊕"/"noMerge")
    public Result<User> noMerge(Integer userId) {
        return Result.data(userWrapBatchService.queryUserNoMerge(userId));
    }
}

```

正常情况下会走 noMerge 接口直接根据 userid 查询用户的信息，但是流量高的情况下会在网关中将请求路径转向合并接口对应的路径 getMergeUser。

```

package ljl.bilibili.user_center.service;

import ljl.bilibili.entity.user_center.user_info.User;
import lombok.Data;

import java.util.concurrent.CompletableFuture;
@Data 9个用法
public class Request {
    // 请求id 唯一
    String requestId;
    // 参数
    Integer userId;
    //TODO Java 8 的 CompletableFuture 并没有 timeout 机制
    CompletableFuture<User> completableFuture;

}

```

合并请求时会用到的自定义请求类，将用户 id 和一个 completablefuture 对象绑定到一起

```

public User queryUser(Integer userId) { 1个用法
    Request request = new Request();
    // 这里用UUID做请求id
    request.requestId = UUID.randomUUID().toString().replace( target: "-", replacement: "");
    request.userId = userId;
    CompletableFuture<User> future = new CompletableFuture<>();
    request.completableFuture = future;
    //将对象传入队列
    queue.offer(request);
    //如果这时候没完成赋值,那么就会阻塞,直到能够拿到值
    try {
        return future.get();
    } catch (InterruptedException e) {
        e.printStackTrace();
    } catch (ExecutionException e) {
        e.printStackTrace();
    }
    return null;
}

public User queryUserNoMerge(Integer userId) { 1个用法
    return userMapper.selectById(userId);
}
}

```

合并请求和不合并请求对应的方法

合并请求时会创建一个新的请求对象, 生成一个随机请求 id, 创建一个用于获取用户信息的 completablefuture 对象, 将该对象传入队列后阻塞主线程直到 completablefuture 对象调用了 complete 方法传入该 userid 对应的用户对象, 将该用户对象做返回值返回调用 queryUser 方法的接口。

```

@PostConstruct
public void init() {
    //定时任务线程池,创建一个支持定时、周期性或延时任务的限定线程数目(这里传入的是1)的线程池
    ScheduledExecutorService scheduledExecutorService = Executors.newScheduledThreadPool( corePoolSize: 1);

    scheduledExecutorService.scheduleAtFixedRate(() -> {
        int size = queue.size();
        //如果队列没数据,表示这段时间没有请求,直接返回
        if (size == 0) {
            return;
        }
        List<Request> list = new ArrayList<>();
        System.out.println("合并了 [" + size + "] 个请求");
        //将队列的请求消费到一个集合保存
        for (int i = 0; i < size; i++) {
            // 后面的SQL语句是有长度限制的, 所以还要做限制每次批量的数量,超过最大任务数,等下次执行
            if (i < MAX_TASK_NUM) {
                list.add(queue.poll());
            }
        }
        //拿到我们需要去数据库查询的特征,保存为集合
        List<Request> userReqs = new ArrayList<>();
        for (Request request : list) {
            userReqs.add(request);
        }
        //将参数传入service处理, 这里是本地服务, 也可以把userService 看成RPC之类的远程调用
        Map<String, User> response = userService.queryUserByIdBatch(userReqs);
    }, 0, 500, TimeUnit.MILLISECONDS);
}

```

在 springboot 启动时创建一个定时线程池，间隔 500ms 执行一次，每次执行时从队列中取出上次执行到这次执行的 500ms 内聚集的自定义请求对象集合

```

public Map<String, User> queryUserByIdBatch(List<Request> userReqs) { 1个用法
    List<Integer> userIds = userReqs.stream().map(Request::getUserId).collect(Collectors.toList());
    QueryWrapper<User> queryWrapper = new QueryWrapper<>();
    // 用in语句合并成一条SQL, 避免多次请求数据库的IO
    queryWrapper.in( column: "id", userIds);
    List<User> users = userMapper.selectList(queryWrapper);
    Map<Integer, List<User>> userGroup = users.stream().collect(Collectors.groupingBy(User::getId));
    HashMap<String, User> result = new HashMap<>();
    userReqs.forEach(val -> {
        List<User> usersList = userGroup.get(val.getUserId());
        if (!CollectionUtils.isEmpty(usersList)) {
            result.put(val.getRequestId(), usersList.get(0));
        } else {
            // 表示没数据
            result.put(val.getRequestId(), null);
        }
    });
    return result;
}
}

```

用这些自定义请求对象集合得到 userid 集合后批量查询得到用户集合，再将该集合按 userid 聚合成 userid 做键，用户信息集合做值的 map，再对该自定义请求对象集合结合 map 进行遍历，得到某次请求 id 做键用户信息对象做值的 map

```
//将参数传入service处理，这里是本地服务，也可以把UserService 看成RPC之类的远程调用
Map<String, User> response = userService.queryUserByIdBatch(userReqs);
//将处理结果返回各自的请求
for (Request request : list) {
    User result = response.get(request.requestId);
    request.completableFuture.complete(result);    //completableFuture.complete方法完成赋值，这一步执行完毕，下面future
    ()阻塞的请求可以继续执行了
}
}, initialDelay: 100, period: 10, TimeUnit.MILLISECONDS);
//scheduleAtFixedRate是周期性执行 schedule是延迟执行 initialDelay是初始延迟 period是周期间隔 后面是单位
//这里初始化后100毫秒后执行，周期性执行10毫秒执行一次
}
```

得到该 map 后遍历自定义请求对象，拿到该自定义请求对象的用户信息对象后调用该自定义请求对象的 completablefuture 对象的 complete 方法将用户信息对象放入进去，执行到这一步原先的请求就不会再阻塞而会返回给前端应该得到的用户信息。但问题在于 completablefuture 是没有超时机制的，也就是说前端有可能一直阻塞着，因此做了优化

```
package ljl.bilibili.user_center.service;

import ljl.bilibili.entity.user_center.user_info.User;
import lombok.Data;

import java.util.concurrent.LinkedBlockingQueue;

@Data 9个用法
public class Request {
    // 请求id 唯一
    String requestId;
    // 参数
    Integer userId;
    //TODO Java 8 的 CompletableFuture 并没有 timeout 机制
    LinkedBlockingQueue<User> usersQueue;
}
```

原先的用于得到用户信息对象的 completablefuture 换成了 LinkedBlockingQueue


```

        Map<String, User> response = usersService.queryUserByIdBatch(userReqs);
        for (Request userReq : userReqs) {
            // 这里再把结果放到队列里
            User users = response.get(userReq.getRequestId());
            userReq.usersQueue.offer(users);
            userReq.completableFuture.complete(result);
        }
    }

```

相比原先将用户信息对象放入 completablefuture 中, 现在是将用户信息对象放入 LinkedBlockingQueue 对象中

```

public User queryUser(Integer userId) { 1个用法
    Request request = new Request();
    // 这里用UUID做请求id
    request.requestId = UUID.randomUUID().toString().replace(" ", "");
    request.userId = userId;
    LinkedBlockingQueue<User> usersQueue = new LinkedBlockingQueue<>();
    request.usersQueue = usersQueue;
    //将对象传入队列
    queue.offer(request);
    //取出元素时, 如果队列为空, 给定阻塞多少毫秒再队列取值, 这里是3秒
    try {
        return usersQueue.poll( timeout: 3000, TimeUnit.MILLISECONDS);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
    return null;
}

```

相比原来一直阻塞, 现在换成了设置超时时间, 队列中一旦有元素就会取出并且返回前端, 但是 3s 内没有取出元素则直接返回空值给前端, 避免了一直等待的问题。